Accident Severity Prediction and Hotspot Analysis of US Road Accidents Using Spark SQL and MLlib on a Hadoop Cluster

Ritvik Kar, Dhruv Abhijit Desai, Mithil S
Department of Data Science Engineering
Manipal Institute of Technology, Manipal, India
Ritvik Kar: 240968198 Dhruv Abhijit Desai: 240968204 Mithil S: 240968160

***Abstract*—Road accident records now run to millions of rows that mix location, time, weather and road-feature attributes, which makes repeated cleaning and model training slow on a single machine. This project built a three-node Hadoop 3.3.6 cluster (one NameNode/ResourceManager and two DataNodes/NodeManagers) from Ubuntu VirtualBox machines spread over two laptops and linked with Tailscale. The US-Accidents dataset (7,728,394 records, about 3 GB as CSV) was stored in HDFS and processed with PySpark 3.5.1 on YARN using Spark SQL/DataFrames and Spark MLlib. After cleaning, 7,692,844 records remained. Spark SQL aggregations identified accident hotspots by state, city, hour, weekday, season, weather and road feature. Logistic Regression and Random Forest classifiers were trained to predict severity (traffic impact, 1 to 4) with class weights and leakage control. Logistic Regression with square-root weights was the best overall model (weighted F1 0.751 against 0.708 for the majority-class baseline), but rare severe classes remained hard to predict. Running the same workload with 1, 2 and 3 executors gave a speed-up of 1.53x with two executors, while a third executor did not help on this small cluster.**

***Index Terms*—Big data analytics, Hadoop, HDFS, Apache Spark, Spark SQL, MLlib, accident severity prediction, hotspot analysis.**

I. Introduction

Road accidents cause loss of life, injury and economic cost, and knowing when, where and under which conditions severe accidents occur supports traffic-safety planning. Modern accident databases combine geographic position, timestamps, weather observations and road-infrastructure indicators for millions of events collected over several years. At this size, loading, cleaning, aggregating and training models on a single machine becomes slow, and iterative analysis is impractical.

Distributed storage (HDFS) and in-memory distributed processing (Apache Spark running on YARN) allow such data to be handled in parallel across several machines. This project built an end-to-end pipeline on a multi-node Hadoop cluster, from raw CSV ingestion to analysis, prediction and a scaling experiment.

A. Problem Statement

The project analysed a countrywide US traffic accident dataset stored in HDFS to (a) identify spatial and temporal accident hotspots and the weather and road conditions most associated with severe accidents, and (b) build and compare machine learning models in Spark MLlib that predict accident severity from time, location, weather and road-feature attributes. All processing was executed on a multi-node Hadoop cluster with Spark running on YARN.

B. Objectives

1) Set up a multi-node Hadoop cluster (one NameNode/ResourceManager and two DataNodes/NodeManagers across two laptops) and load the full dataset (7 million+ records) into HDFS.
2) Clean and transform the data using Spark DataFrames/SQL and engineer time and weather features (hour, day of week, season, weather group).
3) Identify the top accident-prone states and cities, the peak hours, and the weather conditions linked to severe accidents using Spark SQL aggregations and charts.
4) Train and compare MLlib classifiers (Logistic Regression and Random Forest) for severity prediction with imbalance-aware metrics and a confusion matrix.
5) Measure job runtime on YARN with 1, 2 and 3 executors.

C. Why Big Data Tools Are Needed

The data holds millions of records with geospatial, temporal and categorical attributes. Repeated cleaning, aggregation and model training on this volume is slow on one machine. HDFS stores the data across nodes and Spark processes it in parallel, which keeps iterative analysis practical.

II. Dataset Description

A. Source and Size

The **US-Accidents: A Countrywide Traffic Accident Dataset** was published on Kaggle by S. Moosavi et al. [1]–[3]. Table I summarises its main properties.

B. Attribute Groups

The 46 raw attributes fall into the groups listed in Table II. The full schema printed by the ingestion job is shown in the console output in the figures below.

TABLE I: Dataset summary

Property	Value
Name	US-Accidents: A Countrywide Traffic Accident Dataset
Source	Kaggle (S. Moosavi et al.)
Records	7,728,394 raw; 7,692,844 after cleaning
Attributes	46 raw; 44 after cleaning and feature engineering
Size	about 2.8 GB as CSV; 732 MB as Parquet
Period	2016 to March 2023, contiguous United States
License	CC BY-NC-SA 4.0 (research use, attribution, non-commercial)

TABLE II: Attribute groups of the dataset

Group	Attributes
Identification	ID, Source, Severity
Location	Start_Lat, Start_Lng, End_Lat, End_Lng, Street, City, County, State, Zipcode, Country, Timezone, Airport_Code
Time	Start_Time, End_Time, Weather_Timestamp, Sunrise_Sunset, Civil_Twilight, Nautical_ Twilight, Astronomical_Twilight
Weather	Temperature_F, Wind_Chill_F, Humidity, Pressure_in, Visibility_mi, Wind_Direction, Wind_Speed_mph, Precipitation_in, Weather_Condition
Road features (13 flags)	Amenity, Bump, Crossing, Give_Way, Junction, No_Exit, Railway, Roundabout, Station, Stop, Traffic_Calming, Traffic_ Signal, Turning_Loop
Other	Distance_mi, Description



Fig. 1: Schema printed by `ingest.py` (part 1).

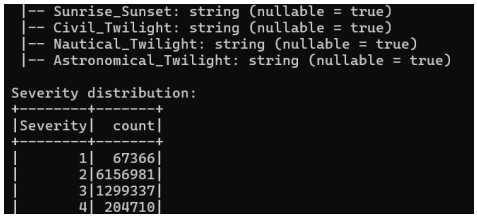


Fig. 2: Severity counts and schema (part 2) printed by `ingest.py`.

C. Meaning of Severity

Severity takes values 1 to 4 and measures the *impact of the accident on traffic* (length of the delay), not injury severity. About 80% of accidents have severity 2, so the classes are strongly imbalanced and plain accuracy is a misleading metric.

D. Why the Dataset Qualifies as Big Data

With 7.7 million records and about 3 GB of CSV, the volume exceeds the 500 MB minimum in the guidelines. The mix of geospatial, temporal and categorical attributes makes repeated cleaning, aggregation and model training slow on one machine, so the dataset benefits from HDFS storage and distributed Spark processing.

E. License and Ethics

The dataset is released under CC BY-NC-SA 4.0 and was used only for academic, non-commercial work with attribution. It describes accident events and contains no names or personal identifiers (see the schema above); the free-text `Description` and `Street` columns were dropped during cleaning.

III. Architecture and Cluster Configuration

A. Cluster Design

The cluster consisted of three Ubuntu Server 22.04 virtual machines run under VirtualBox on two laptops. Laptop A (Ritvik) hosted the `master` and `worker1` VMs; Laptop B (Dhruv) hosted `worker2`. Figure 3 shows the layout.

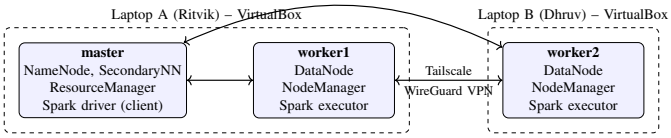


Fig. 3: Cluster architecture across two laptops, linked with Tailscale.

B. Node Configuration

TABLE III: Node and software configuration

Item	Setting
Virtualisation	VirtualBox, 3 Ubuntu Server 22.04 VMs
VM size	2 vCPU, 3 GB RAM, 20 GB disk (each)
master	NameNode, SecondaryNameNode, ResourceManager, Spark driver (client mode)
worker1, worker2	DataNode (HDFS), NodeManager (YARN), Spark executors
Java	OpenJDK 11
Hadoop	3.3.6, HDFS replication factor 2
Spark	3.5.1, PySpark, YARN client mode
YARN	2 NodeManagers $\times$ 2048 MB
Spark executors	2 $\times$ (768 MB + 256 MB overhead)
Networking	Tailscale (free tier)

C. HDFS and YARN Configuration

The configuration files are kept in the `cluster/` folder of the repository: `core-site.xml`, `hdfs-site.xml`, `mapred-site.xml`, `yarn-site.xml`, `workers` and `spark-defaults.conf`. HDFS was set to a replication factor of 2, so each worker holds a copy of every block; the 3 GB CSV therefore occupies about 6 GB of raw disk space. YARN schedules Spark executors on the two NodeManagers,

TABLE IV: Key Configuration Properties

File	Property	Value
core-site.xml	fs.defaultFS	hdfs://master:9000
hdfs-site.xml	dfs.replication	2
hdfs-site.xml	NameNode / DataNode directories	/opt/hadoop/data/nn, /opt/hadoop/data/dn
yarn-site.xml	yarn.resourcemanager.hostname	master
yarn-site.xml	yarn.nodemanager.resource.memory-mb	2048
yarn-site.xml	Scheduler min / max allocation	256 / 2048 MB
yarn-site.xml	yarn.nodemanager.vmem-check-enabled	false
mapred-site.xml	mapreduce.framework.name	yarn (MapReduce classpath set)
spark-defaults.conf	spark.master, spark.submit.deployMode	yarn, client
spark-defaults.conf	spark.driver.memory	1g
spark-defaults.conf	spark.executor.instances / cores	2 / 1
spark-defaults.conf	Executor memory + memoryOverhead	768m + 256m
spark-defaults.conf	spark.yarn.am.memory + overhead	512m + 256m
spark-defaults.conf	spark.sql.shuffle.partitions	16

each limited to 2048 MB of container memory, which gives 4 GB of container memory for the whole cluster.

Table IV lists the key properties; the complete files are in Appendix A.

D. Why Tailscale Was Needed

On a phone hotspot and on campus Wi-Fi, the two laptops could not reach each other directly. Tailscale (a WireGuard-based VPN) gave every VM a stable address that works on any network, so the NameNode, ResourceManager and workers could communicate as one cluster [7].

E. Engineering Challenges

- The laptops could not reach each other on hotspot or campus Wi-Fi; this was solved with Tailscale.
- YARN could not find the MapReduce classes; this was fixed in `mapred-site.xml`.
- The 2 GB NodeManagers imposed memory limits; the Spark executor sizes were tuned to fit.

IV. Methodology and Spark Code Explanation

Every job ran on YARN in client mode, read its input from HDFS and wrote its output to HDFS. The data flow was:

`accidents/ (CSV)` → `accidents_parquet/` → `accidents_clean/` → `results/*.csv` + `charts/*.png`

A. Ingestion (`ingest.py`)

The job read the 3 GB CSV from HDFS with schema inference, renamed every column so that it contained only letters, digits and underscores (Parquet cannot store names with spaces or brackets), and then printed the schema, the severity distribution and the null count of every column. Finally it wrote the data as Parquet. Parquet is columnar and compressed: the output was 732 MB, about four times smaller than the CSV, and later jobs read only the columns they need.

```
df = spark.read.csv(src, header=True, inferSchema=True)
for c in df.columns:
   df = df.withColumnRenamed(c, re.sub(r"[^0-9a-zA-Z]+", "
        _", c).strip("_"))
df.write.mode("overwrite").parquet(dst)
```

B. Cleaning and Feature Engineering (`clean.py`)

The cleaning job reduced 7,728,394 rows to 7,692,844 rows and proceeded in six steps:

1) **Deduplication and column removal.** Duplicate `ID` values were dropped. Columns that were mostly empty, constant or free text were removed: End_Lat, End_Lng, Wind_Chill_F, Description, Street, Country, Airport_Code, Weather_Timestamp, Zipcode and Source.
2) **Impossible values.** Rows were removed when a measurement fell outside a physical range (for example temperature outside -60 to $130\,^{\circ}$F, humidity outside 0 to 100, pressure outside 20 to 35 inHg). Null values were kept at this stage and imputed later.
3) **Time features.** Hour, day of week, month, year, a weekend flag, a season (Winter, Spring, Summer, Autumn) and the accident duration in minutes were derived from `Start_Time` and `End_Time`. Rows with a non-positive duration or a duration over 1440 minutes were removed.
4) **Imputation.** Missing numeric weather values were filled with the approximate median (`approxQuantile`); missing precipitation was set to 0; missing categorical values were set to "Unknown".
5) **Weather grouping.** The 100+ weather labels were mapped with regular expressions to Snow/Ice, Rain/Storm, Fog/Haze, Cloudy, Clear, Other and Unknown.
6) **Boolean encoding.** The 13 road-feature flags were cast to 0/1 so that MLlib can use them.

```
df = df.withColumn("weather_group",
   F.when(wc == "unknown", "Unknown")
    .when(wc.rlike("snow|sleet|ice|wintry|freezing"), "
         Snow/Ice")
    .when(wc.rlike("rain|drizzle|shower|thunder|storm"), "
         Rain/Storm")
    .when(wc.rlike("fog|mist|haze|smoke"), "Fog/Haze")
    .when(wc.rlike("cloud|overcast"), "Cloudy")
    .when(wc.rlike("clear|fair"), "Clear")
    .otherwise("Other"))
```

C. Spark SQL Analysis (`analysis.py`)

The cleaned Parquet data was registered as a temporary view, and seven Spark SQL queries grouped accidents by state, city, hour, weekday, weather group, season and year. Each query returned the accident count and the *severe share*, defined as the percentage of accidents with severity 3 or 4. A further set of queries compared accidents with and without six road features (traffic signal, junction, crossing, stop, railway, bump). The small result tables were converted to pandas, saved

as CSV files in `results/` and plotted with matplotlib into `charts/`.

```
SELECT State, COUNT(*) AS accidents,
     ROUND(100 * AVG(CASE WHEN Severity >= 3 THEN 1 ELSE 0
          END), 1) AS severe_pct
FROM acc GROUP BY State ORDER BY accidents DESC LIMIT 10
```

D. MLlib Pipeline (`ml.py`, `ml2.py`)

a) Sampling and split.: A stratified sample of 1,540,825 rows (20% of each severity class, seed 42) was drawn and split 80/20 into 1,232,682 training rows and 308,143 test rows. An earlier run (`ml.py`) used a 13% sample of 1,001,088 rows.

b) Features.: Twenty-five numeric features (latitude, longitude, five weather measurements plus precipitation, hour, day of week, month, weekend flag and the 13 road flags) and five categorical features (State, weather_group, season, Sunrise_Sunset, Wind_Direction) were used. Categorical columns were converted with `StringIndexer` and `OneHotEncoder` and combined with the numeric columns by `VectorAssembler`.

c) Leakage control.: Accident duration, distance and year were excluded, because they are known only after the accident or reflect data-collection changes rather than conditions that cause severity.

d) Class weights.: Because severity 2 dominates, each training row received a weight. The *inverse* weight of class k is $w_k = N/(K \cdot n_k)$, where N is the number of training rows, $K = 4$ classes and n_k the class size. The *sqrt* weight is $\sqrt{w_k}$, a milder correction.

e) Resulting weights.: In the first experiment (`ml.py`, 800,531 training rows) the inverse weights were about 28.58 for severity 1, 0.31 for severity 2, 1.48 for severity 3 and 9.62 for severity 4 (`ml_output.txt`), so rare classes were weighted much more heavily than severity 2.

f) Models.: Logistic Regression (multinomial, 30 iterations, $\lambda = 0.01$) and Random Forest (30 trees, maximum depth 8) were trained with each weighting, with and without the month and season features.

g) Metrics.: Accuracy, weighted F1, macro F1, balanced accuracy and per-class recall were computed from the confusion matrix, and compared with a majority-class baseline that always predicts severity 2.

```
lr = LogisticRegression(featuresCol="features", labelCol="
     label",
      weightCol=wc, maxIter=30, regParam=0.01, family="
          multinomial")
rf = RandomForestClassifier(featuresCol="features",
     labelCol="label",
      weightCol=wc, numTrees=30, maxDepth=8, seed=42)
```

E. Executor-Scaling Experiment (`scaling_job.py`)

The same workload of four aggregation queries was run with 1, 2 and 3 executors on YARN (launched by `scripts/run_scaling.sh`), and the elapsed time was recorded. The queries grouped by state and year, by city, by hour/weather/season, and by county after a repartition on state, which forces a shuffle between executors. The better of two runs was reported.



Fig. 4: Row count and severity distribution after ingestion.

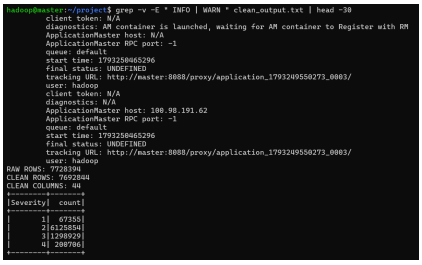


Fig. 5: Output of `clean.py`: row counts, severity and weather-group counts.

V. Results

A. Data Quality and Class Distribution

Ingestion wrote 7,728,394 rows to Parquet. The null counts showed that some columns were largely empty: End_Lat and End_Lng (3,402,762 nulls each), Wind_Chill_F (1,999,019) and Precipitation_in (2,203,586); Wind_Speed_mph had 571,233 nulls and the other weather columns about 140,000 to 177,000. Cleaning reduced the data from 7,728,394 to 7,692,844 rows (35,550 rows removed) and from 46 to 44 columns. About 80% of the records (6,125,854 of 7,692,844) have severity 2 (Table V, Figure 5).

TABLE V: Severity distribution before and after cleaning (`ingest_output.txt`, `clean_output.txt`).

Severity	Raw rows	Clean rows
1	67,366	67,355
2	6,156,981	6,125,854
3	1,299,337	1,298,929
4	204,710	200,706
Total	7,728,394	7,692,844

After weather grouping, the clean data contained 3,387,379 Clear, 3,151,719 Cloudy, 598,602 Rain/Storm, 200,361 Fog/Haze, 178,041 Snow/Ice, 172,560 Unknown and 4,182 Other records.

B. Hotspots and Temporal Patterns

a) States and cities.: California recorded the most accidents (1.74 million), followed by Florida and Texas. The severe share varied widely: New York 23.2%, Virginia 22.9% and Texas 21.9%, against 8.5% in Oregon. The most affected cities were Miami, Houston and Los Angeles (Tables VI, VII, Figure 6).

b) Time of day.: Accident volume peaks during the morning (07:00–08:00) and evening (16:00–17:00) commutes, but the severe share is highest around 04:00 and between 18:00 and 20:00 (about 21.5%) (Figures 7, 8).

TABLE VI: Top 10 states by accident count (`results/top_states.csv`).

State	Accidents	Severe share (%)
CA	1739361	16.4
FL	876742	13.4
TX	582423	21.9
SC	382002	11.9
NY	347333	23.2
NC	337802	11.8
VA	303112	22.9
PA	296119	15.8
MN	191828	16.0
OR	175772	8.5

TABLE VII: Top 10 cities by accident count (`results/top_cities.csv`).

City	State	Accidents	Severe share (%)
Miami	FL	185639	9.1
Houston	TX	169340	18.0
Los Angeles	CA	156249	22.9
Charlotte	NC	138313	5.6
Dallas	TX	130228	26.2
Orlando	FL	109063	6.5
Austin	TX	96538	9.2
Raleigh	NC	86051	9.9
Nashville	TN	72641	15.1
Baton Rouge	LA	70999	9.5

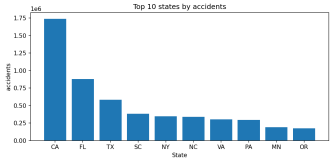


Fig. 6: Top 10 states by number of accidents.

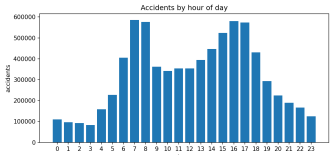


Fig. 7: Accidents by hour of day.

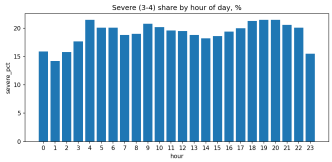


Fig. 8: Severe (severity 3–4) share by hour of day.

c) Weekdays.: Weekends had fewer accidents (Saturday 664k, Sunday 559k, against about 1.3 million on a weekday) but a higher severe share (21.1% and 22.4% against about 19%).

d) Season.: Summer had a severe share of 23.1% against 16.5% in winter.

TABLE VIII: Accidents and severe share by weekday (1 = Sunday; `results/by_weekday.csv`).

Day of week	Accidents	Severe share (%)
1	558763	22.4
2	1205842	19.3
3	1285601	19.0
4	1306830	18.9
5	1310531	19.0
6	1360797	19.2
7	664480	21.1

TABLE IX: Accidents and severe share by season (`results/by_season.csv`).

Season	Accidents	Severe share (%)
Winter	2245934	16.5
Autumn	2076484	18.4
Spring	1693042	21.3
Summer	1677384	23.1

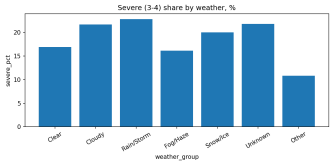


Fig. 9: Severe share by weather group.

e) Weather.: Rain and storms had the highest severe share (22.8%) against 16.9% in clear weather. Fog and haze (16.1%) was lower, possibly because drivers slow down.

TABLE X: Accidents and severe share by weather group (`results/by_weather.csv`).

Weather group	Accidents	Severe share (%)
Clear	3387379	16.9
Cloudy	3151719	21.7
Rain/Storm	598602	22.8
Fog/Haze	200361	16.1
Snow/Ice	178041	20.0
Unknown	172560	21.8
Other	4182	10.8

f) Road features.: Junctions were riskier (26.8% severe against 18.9% without), while crossings (7.0%) and bumps (9.7%) were associated with milder accidents, possibly because they occur on slower urban roads.

g) Yearly trend.: The accident count per year (Figure 10) and the yearly severity trend reflect how the data was collected over time rather than real changes in road safety; for this reason the year was excluded from the models.

h) Caution.: These are counts of recorded accidents, not risk rates, because no traffic-volume data was available, and they show associations, not causes.

C. Severity Prediction Results

Table XII compares all configurations on the test set (without month and season features). Figure 13 shows the same

TABLE XI: Severe share with and without each road feature (`results/infrastructure.csv`).

Feature	Present	Accidents	Severe share (%)
Bump	0	7689366	19.5
Bump	1	3478	9.7
Crossing	0	6822591	21.1
Crossing	1	870253	7.0
Junction	0	7123558	18.9
Junction	1	569286	26.8
Railway	0	7626184	19.5
Railway	1	66660	15.1
Stop	0	7480237	19.9
Stop	1	212607	6.2
Traffic_Signal	0	6551704	21.2
Traffic_Signal	1	1141140	9.5

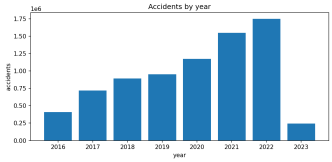


Fig. 10: Accidents by year.

```
hadoop@master:~/project$ grep -v -E " INFO | WARN " analysis_output.txt | head -50
        client token: N/A
        diagnostics: AM container is launched, waiting for AM container to Register with RM
        ApplicationMaster host: N/A
        ApplicationMaster RPC port: -1
        queue: default
        start time: 1793250762648
        final status: UNDEFINED
        tracking URL: http://master:8088/proxy/application_1793249550273_0004/
        user: hadoop
        client token: N/A
        diagnostics: N/A
        ApplicationMaster host: 100.100.110.62
        ApplicationMaster RPC port: -1
        queue: default
        start time: 1793250762648
        final status: UNDEFINED
        tracking URL: http://master:8088/proxy/application_1793249550273_0004/
        user: hadoop

== top_states
State  accidents  severe_pct
   CA    1739361        16.4
   FL     876742        13.4
   TX     582423        21.9
   SC     382002        11.9
   NY     347333        23.2
   NC     337892        11.8
   VA     303112        22.9
   PA     296119        15.8
   MN     191828        16.0
   OR     175772         8.5
```

Fig. 11: Console output of `analysis.py` (part 1).

```
== top_cities
       City State  accidents  severe_pct
      Miami    FL     185639         9.1
    Houston    TX     169340        18.0
Los Angeles    CA     156249        22.9
  Charlotte    NC     138313         5.6
     Dallas    TX     130228        26.2
    Orlando    FL     109063         6.5
     Austin    TX      96538         9.2
    Raleigh    NC      86051         9.9
  Nashville    TN      72641        15.1
Baton Rouge    LA      70999         9.5

== by_hour
 hour  accidents  severe_pct
    0     110142        15.9
    1      96046        14.2
    2      92285        15.8
```

Fig. 12: Console output of `analysis.py` (part 2).

comparison.

a) Best model.: Logistic Regression with sqrt weights was the best all-round model: weighted F1 0.751 against 0.708, and macro F1 0.334 against 0.222 for the baseline. Accuracy fell slightly (0.778 against 0.798), which is expected because the model sacrifices some majority-class accuracy to detect rarer classes.

b) Weights are a trade-off.: Inverse weights found more rare severe cases (balanced accuracy up to 0.51) but produced many false alarms and low accuracy; sqrt weights kept accuracy higher.

TABLE XII: Model comparison on the test set (308,143 rows; `results/ml_experiments.csv`).

Model	Wts	Acc.	W-F1	M-F1	Bal.	R1	R2	R3	R4	Time (s)
Baseline	–	0.798	0.708	0.222	0.250	0.000	1.000	0.000	0.000	–
LR	sqrt	0.778	0.751	0.334	0.319	0.061	0.917	0.265	0.035	18.2
LR	inverse	0.372	0.462	0.256	0.506	0.682	0.347	0.450	0.546	19.2
RF	sqrt	0.799	0.713	0.237	0.258	0.018	0.998	0.013	0.000	127.1
RF	inverse	0.287	0.351	0.218	0.510	0.711	0.225	0.512	0.593	128.9

LR = Logistic Regression, RF = Random Forest, Wts = class weighting, W-F1 = weighted F1, M-F1 = macro F1, Bal. = balanced accuracy, R1–R4 = recall of severity 1–4, Time = training time. Features without month and season.



Fig. 13: Model comparison on the test set (without month/season).



Fig. 14: Row-normalised confusion matrix of Logistic Regression (sqrt weights).

c) Random Forest.: Random Forest did not outperform Logistic Regression. The likely reasons are the shallow trees (depth 8) and the small 1 GB executors; it also trained about six times slower (about 127 s against 18–21 s for Logistic Regression).

d) Timing features.: Removing month and season left the weighted F1 of Logistic Regression (sqrt) practically unchanged (0.7511 with, 0.7510 without), so the models did not simply learn data-collection timing.

e) Per-class behaviour.: Severity 2 was recognised well (92% recall), but severity 3 only 26% (73% of severity-3 cases were predicted as 2), severity 4 only 3% and severity 1 6%.

f) Feature importance.: Random Forest importance (Figure 15) was highest for road infrastructure and location: traffic signals, crossings, pressure, temperature, latitude/longitude and hour. Importance does not imply causation.

g) Conclusion on modelling.: Weather, time and road features explain only part of the traffic impact of an accident; traffic volume and speed data would be needed for better prediction.

D. Executor Scaling on YARN

Going from one to two executors reduced the runtime from 28.7 s to 18.8 s, a speed-up of 1.53x (about 76% efficiency),

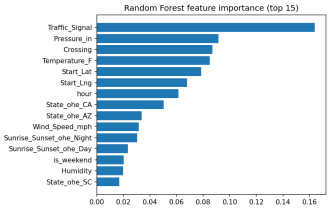


Fig. 15: Random Forest feature importance (top 15).

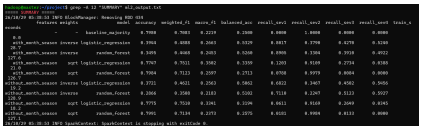


Fig. 16: Summary table printed by `ml2.py`.

TABLE XIII: Runtime of the scaling workload on YARN (two runs per setting; `results/scaling.csv`).

Executors	Run 1 (s)	Run 2 (s)	Better run, speed-up
1	35.3	28.7	28.7 s, 1.00x
2	19.7	18.8	18.8 s, 1.53x
3	23.4	23.1	23.1 s, 1.24x

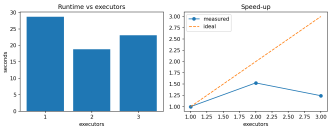


Fig. 17: Runtime against number of executors.

```
hadoop@master:~/project$ cat results/scaling.csv
requested,actual,run,seconds
1,1,1,35.3
1,1,2,28.7
2,2,1,19.7
2,2,2,18.8
3,3,1,23.4
3,3,2,23.1
```

Fig. 18: Console output of the scaling runs.

with one executor per worker. Three executors were slower (23.1 s). The probable causes are that two executors then shared one 2-vCPU VM, that the job is short (about 20 s) so scheduling, JVM start-up and shuffling over the VPN dominate, and that the cluster has only 4 GB of container memory. Distribution therefore helps, but the gain flattens quickly on a small cluster; more nodes or larger workloads are needed to scale further.

E. Cluster Evidence

The screenshots in this section show that the multi-node cluster was running and that the jobs were executed on YARN.

a) Daemons and network.: The `jps` output on each VM lists the Hadoop daemons, and Tailscale connected the VMs across both laptops.

b) HDFS.: Two live DataNodes were reported, the dataset folders were stored in HDFS, and the file blocks were placed on both DataNodes.



Fig. 19: `jps` on master: NameNode, SecondaryNameNode and ResourceManager.

```
hadoop@master:~$ ssh worker1 "hostname; jps"
worker1
1138 DataNode
1291 NodeManager
23660 Jps
```

Fig. 20: `jps` on worker1: DataNode and NodeManager.

```
hadoop@master:~$ ssh worker2 "hostname; jps"
worker2
1448 DataNode
1598 NodeManager
1870 Jps
```

Fig. 21: `jps` on worker2: DataNode and NodeManager.



Fig. 22: Tailscale status of the cluster VMs.

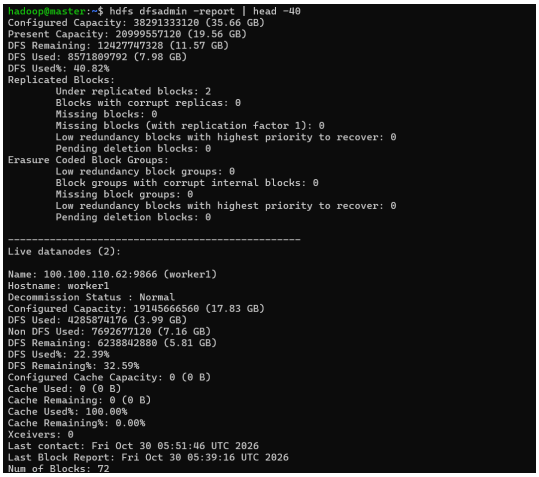


Fig. 23: `hdfs dfsadmin -report` showing two live DataNodes.

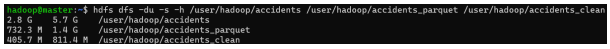


Fig. 24: HDFS folder sizes (CSV and Parquet).

c) YARN.: Both NodeManagers were registered with the ResourceManager and the Spark applications appeared in the YARN UI.

d) Spark.: The Spark history server listed the completed applications.

VI. Conclusion, Limitations and Future Work

A. Conclusion

A working three-node Hadoop cluster (HDFS and YARN) was built across two laptops, the 7.7 million-record dataset was loaded into HDFS, and an end-to-end Spark pipeline ran

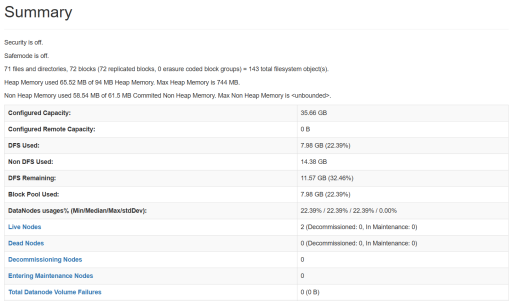


Fig. 25: HDFS NameNode web UI: overview.

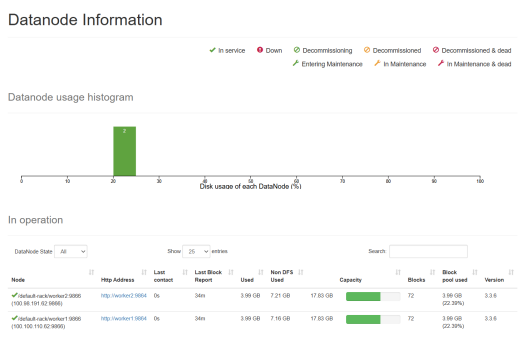


Fig. 26: HDFS web UI: DataNodes page.

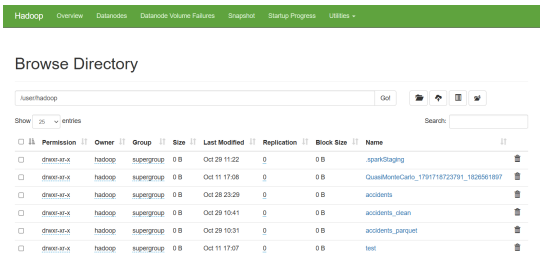


Fig. 27: HDFS file browser showing the dataset folders.

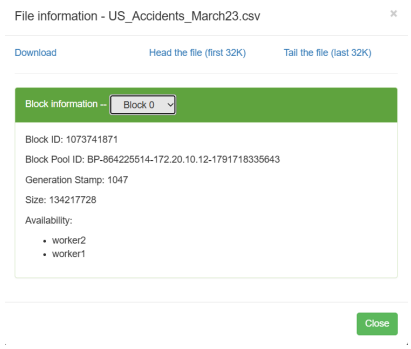


Fig. 28: Block locations of the dataset file on the two DataNodes.

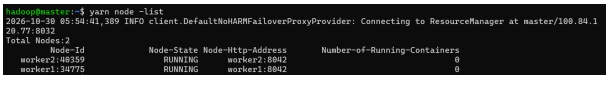


Fig. 29: `yarn node -list` showing both NodeManagers.

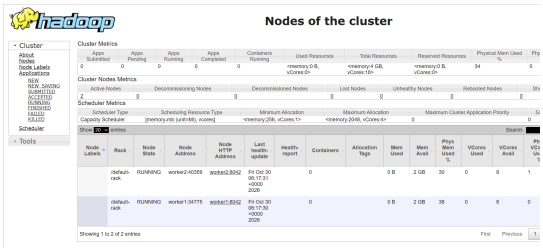


Fig. 30: YARN UI: Nodes page with two NodeManagers running.

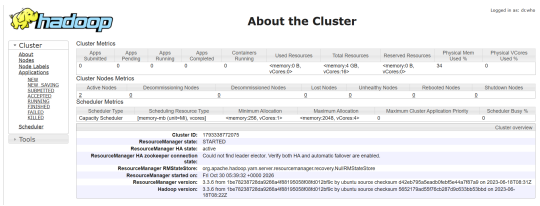


Fig. 31: YARN ResourceManager UI: cluster overview and applications.

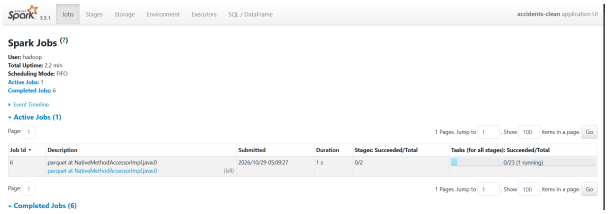


Fig. 32: Spark history server: applications run on YARN (part 1).

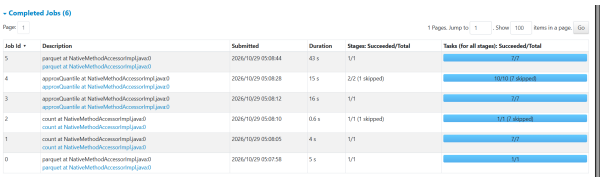


Fig. 33: Spark history server: completed applications (part 2).

on YARN using Spark SQL/DataFrames and MLlib. The analysis showed clear spatial, temporal, weather and road-feature patterns, such as higher severe shares at junctions, in rain, in summer and at weekends. The classifiers improved modestly over the majority-class baseline (weighted F1 0.751 against 0.708), and the scaling experiment gave a 1.53x speed-up with two executors. The results were compared with a baseline and reported honestly, including the models' weakness on rare severe classes.

B. Limitations

- Severity measures traffic impact, not injury.
- Counts are not rates, since no traffic-volume data was used.
- The yearly severity trend is a data-collection artefact, so year was excluded.

- Models used a 1.5 million-row sample and shallow trees on small 1 GB executors.
- The cluster is small and VPN-linked, so the scaling results show a trend, not a precise benchmark.

C. Future Work

- Train on the full dataset with larger executors or more nodes.
- Try gradient-boosted trees and a time-based train/test split.
- Add traffic volume, speed limits and road-network data.
- Use Spark Structured Streaming for near-real-time hotspot alerts.
- Build map-based hotspot visualisations.

REFERENCES

[1] S. Moosavi, M. H. Samavatian, S. Parthasarathy and R. Ramnath, "A Countrywide Traffic Accident Dataset," arXiv:1906.05409, 2019.

[2] S. Moosavi et al., "Accident Risk Prediction based on Heterogeneous Sparse Data: New Dataset and Insights," ACM SIGSPATIAL, 2019.

[3] US-Accidents dataset page, Kaggle: https://www.kaggle.com/datasets/sobhanmoosavi/us-accidents

[4] Apache Hadoop 3.3.6 documentation, https://hadoop.apache.org/docs/r3.3.6/

[5] Apache Spark 3.5.1 documentation, https://spark.apache.org/docs/3.5.1/

[6] Project source code and results: https://github.com/rit-vik/Accident-Severity-Prediction-and-Hotspot-Analysis-of-US-Road-Accidents-BDA-Project-

[7] Tailscale documentation, https://tailscale.com/kb

APPENDIX

A. core-site.xml

```
<?xml version="1.0"?>
<configuration>
  <property><name>fs.defaultFS</name><value>hdfs://master
       :9000</value></property>
</configuration>
```

B. hdfs-site.xml

```
<?xml version="1.0"?>
<configuration>
  <property><name>dfs.replication</name><value>2</value></
       property>
  <property><name>dfs.namenode.name.dir</name><value>/opt/
       hadoop/data/nn</value></property>
  <property><name>dfs.datanode.data.dir</name><value>/opt/
       hadoop/data/dn</value></property>
</configuration>
```

C. mapred-site.xml

```
<?xml version="1.0"?>
<configuration>
  <property><name>mapreduce.framework.name</name><value>
       yarn</value></property>
  <property><name>yarn.app.mapreduce.am.env</name><value>
       HADOOP_MAPRED_HOME=/opt/hadoop</value></property>
  <property><name>mapreduce.map.env</name><value>
       HADOOP_MAPRED_HOME=/opt/hadoop</value></property>
  <property><name>mapreduce.reduce.env</name><value>
       HADOOP_MAPRED_HOME=/opt/hadoop</value></property>
  <property><name>mapreduce.application.classpath</name><
       value>/opt/hadoop/share/hadoop/mapreduce/*:/opt/
       hadoop/share/hadoop/mapreduce/lib/*</value></
       property>
  <property><name>yarn.app.mapreduce.am.resource.mb</name><
       value>512</value></property>
  <property><name>mapreduce.map.memory.mb</name><value
       >512</value></property>
  <property><name>mapreduce.reduce.memory.mb</name><value
       >512</value></property>
</configuration>
```

D. yarn-site.xml

```
<?xml version="1.0"?>
<configuration>
   <!-- Resource Manager Location Settings -->
   <property>
      <name>yarn.resourcemanager.hostname</name>
      <value>master</value>
   </property>

   <property>
      <name>yarn.nodemanager.hostname</name>
      <value>master</value>
   </property>

   <!-- Core MapReduce & Auxiliary Services -->
   <property>
      <name>yarn.nodemanager.aux-services</name>
      <value>mapreduce_shuffle</value>
   </property>

   <!-- Memory Footprint Allocation Adjustments -->
   <property>
      <name>yarn.nodemanager.resource.memory-mb</name>
      <value>2048</value>
   </property>

   <property>
      <name>yarn.scheduler.minimum-allocation-mb</name>
      <value>256</value>
   </property>

   <property>
      <name>yarn.scheduler.maximum-allocation-mb</name>
      <value>2048</value>
   </property>

   <!-- Virtual Memory Verification Override -->
   <property>
      <name>yarn.nodemanager.vmem-check-enabled</name>
      <value>false</value>
   </property>
</configuration>
```

E. spark-defaults.conf

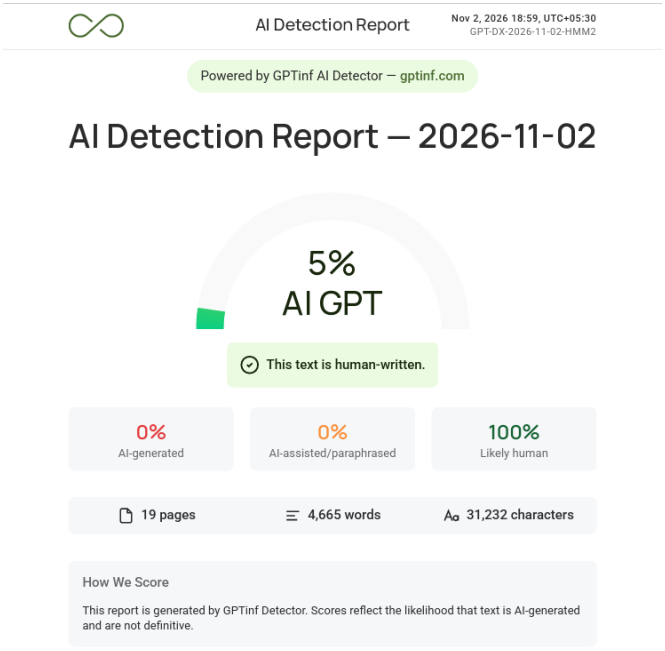
```
spark.master yarn
spark.submit.deployMode client
spark.driver.memory 1g
spark.executor.instances 2
spark.executor.cores 1
spark.executor.memory 768m
spark.executor.memoryOverhead 256m
spark.yarn.am.memory 512m
spark.yarn.am.memoryOverhead 256m
spark.sql.shuffle.partitions 16
```

F. scripts/run_scaling.sh

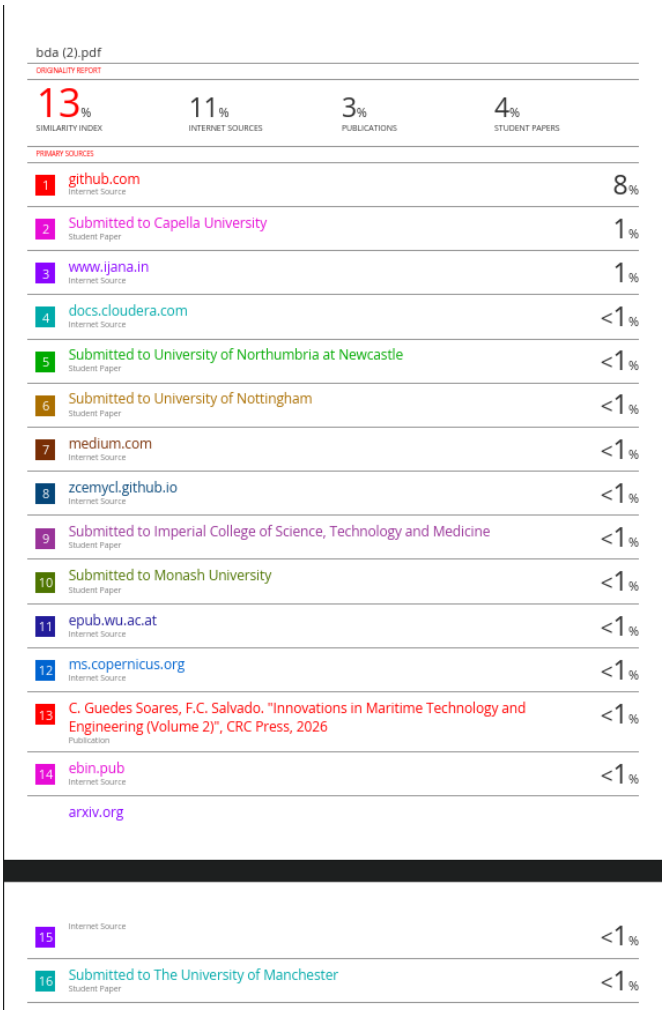
```
#!/bin/bash
# Runs src/scaling_job.py with 1, 2 and 3 executors (2
     runs each) and writes results/scaling.csv
cd "$(dirname "$0")/.."
mkdir -p logs results
echo "requested,actual,run,seconds" > results/scaling.csv
for N in 1 2 3; do
  for RUN in 1 2; do
    OUT=$(spark-submit --conf spark.executor.instances=$N
         --conf spark.dynamicAllocation.enabled=false \
      --conf spark.scheduler.minRegisteredResourcesRatio=1.0
           \
      --conf spark.scheduler.
           maxRegisteredResourcesWaitingTime=180s \
```

```
    src/scaling_job.py 2> logs/scaling_${N}_${RUN}.log |
        grep '^RESULT')
  echo "$N,$(echo $OUT | awk '{print $2}'),$RUN,$(echo
      $OUT | awk '{print $3}')" >> results/scaling.csv
 done
done
cat results/scaling.csv
```

The complete project is available at https://github.com/ritvik/Accident-Severity-Prediction-and-Hotspot-Analysis-of-US-Road-Accidents-BDA-Project-. Source code is provided in the repository folder `src/` (`ingest.py`, `clean.py`, `analysis.py`, `ml.py`, `ml2.py`, `scaling_job.py`, `make_ml_charts.py`) and `scripts/run_scaling.sh`.



(a) AI detection report.



(b) Plagiarism detection report.

Fig. 34: AI detection and plagiarism verification reports.